Reliable RL Scaling Requires Thinking about Prefill-Decode Kernel Mismatch

Yifan Zhang et al.

yifanzhangresearch@gmail.com

August 7, 2026

Abstract

Language-model reinforcement learning is literally on-policy only when the executable distribution used by rollout is the same distribution differentiated by the learner. Equal checkpoints are insufficient: prompt prefill, recurrent decoding, teacher-forced scoring, chunkwise scans, recurrent state updates, cache precision, and reduction order can induce different token probabilities. The mismatch is especially consequential for linear attention and state-space models, where a small state discrepancy can propagate through every later update.

This paper develops four complementary remedies. First, treat kernel mismatch as off-policy data and use the actor-emitted probability in an explicit importance ratio; this covers both prefill–decode mismatch and chunkwise-parallel–recurrent mismatch. Second, make one chunkwise or recurrent execution rule canonical across rollout and differentiable learner replay. TTT and Titans illustrate chunkwise memory-update semantics, while RWKV-7 illustrates a recurrent architecture, but neither removes mismatch unless the same executable rule is actually used on both sides. Third, retain recurrent states, sensitive parameters, and state-update accumulations in higher precision; this reduces numerical drift but is not an equality proof. Fourth, use a fast prefill or parallel path only as a speculative proposal and let the recurrent path perform exact modified rejection sampling and define the accepted policy. We state the mathematical contracts, algorithms, diagnostics, and limitations of each route, and recommend a hybrid design that preserves recurrent actor probabilities while allowing fast learner and proposal kernels.

Project Page: https://github.com/yifanzhang-pro/Pretraining-RL-Science

1 Introduction

The usual definition of an on-policy reinforcement-learning update assumes that the distribution used to score a sampled action is the same distribution that produced it. In a language-model system, however, a checkpoint does not uniquely determine that distribution. The executable path also matters: attention or recurrence kernels, prompt-state construction, cache layout, precision, quantization, parallel layout, reduction order, and the sampling transform can all affect the token probabilities.

Let K denote this execution operator. If $z_\theta^{\mathsf{K}}(h)$ is the logit vector produced at history h, define the effective token policy

$$\pi_\theta^{\mathsf{K}}(\cdot \mid h) = \mathcal{T}_{\mathsf{K}}\left(z_\theta^{\mathsf{K}}(h)\right). \tag{1.1}$$

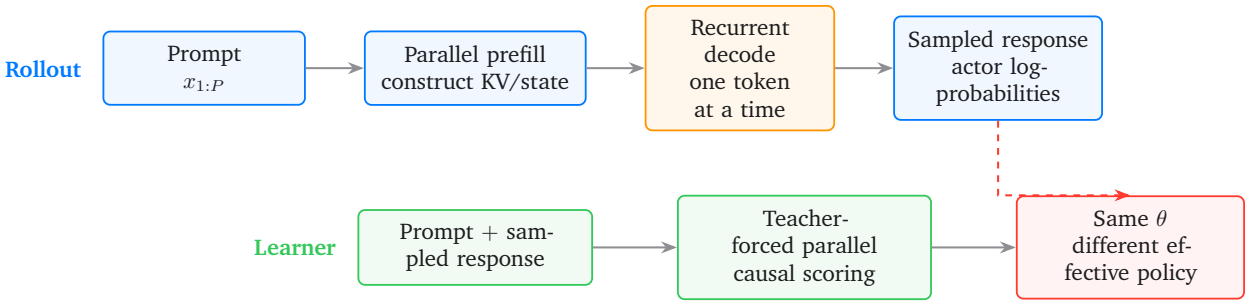

Figure 1 The common actor–learner split. For response tokens after the first generated token, rollout probabilities normally come from recurrent decode, while learner probabilities normally come from a parallel teacher-forced pass.

A record generated and scored at the same parameter vector θ is literally on-policy only if

$$\pi_\theta^{\mathsf{K}_{\text{roll}}}(\cdot \mid h) = \pi_\theta^{\mathsf{K}_{\text{learn}}}(\cdot \mid h) \tag{1.2}$$

for every history reached with positive rollout probability. Equal weights, checkpoint identifiers, token sequences, or model names do not imply Eq. (1.2).

The concrete implementation bug is to sample with K_{roll} but later reconstruct the "old" probability with $\mathsf{K}_{\text{learn}}$. The correct recordwise ratio is

$$\rho_t = \frac{\pi_{\theta_n}^{\mathsf{K}_{\text{learn}}}(a_t \mid h_t)}{\pi_{\theta_v}^{\mathsf{K}_{\text{roll}}}(a_t \mid h_t)}, \tag{1.3}$$

whereas the buggy recomputation uses

$$\widetilde{\rho}_t = \frac{\pi_{\theta_n}^{\mathsf{K}_{\text{learn}}}(a_t \mid h_t)}{\pi_{\theta_v}^{\mathsf{K}_{\text{learn}}}(a_t \mid h_t)}. \tag{1.4}$$

At zero parameter lag, $n = v$, the recomputed ratio in Eq. (1.4) is one by construction. The true ratio in Eq. (1.3) can still differ from one because the actor and learner executed different operators. Recomputing the denominator with the learner engine therefore hides, rather than fixes, the mismatch.

2 Softmax Attention: Prefill and Decode

A standard autoregressive rollout has two phases. First, the prompt is processed with a parallel prefill kernel, which constructs the KV cache and produces the distribution for the first generated token. Later response tokens are produced by one-token recurrent decoding over that cache. The learner typically does something else: it concatenates the prompt and sampled response and evaluates them in one teacher-forced parallel causal pass. Thus, after the first generated token, the behavior probability and learner probability usually come from different kernel families, as shown in Fig. 1.

In exact real arithmetic, parallel causal softmax attention and the corresponding KV-cached recurrence represent the same mathematical function. That algebraic identity is not a finite-precision policy-equality certificate. The implementations can differ in

- tiling and fusion decisions;
- floating-point accumulation and reduction order;
- KV-cache layout and prompt-state construction;
- batch shape and sequence length;
- tensor- and sequence-parallel communication;
- precision, quantization, and numerical rounding.

Any of these can perturb logits and therefore token probabilities. The gap may be negligible in one configuration and large enough to affect reinforcement-learning optimization in another; it must be measured rather than inferred from checkpoint identity (Zhong et al., 2026; Zhang et al., 2025).

The sampled-token log-ratio separates the two sources of mismatch:

$$\log \rho_{t,n,v} = \underbrace{\log \pi_{\theta_n}^{\mathsf{K}_{\text{learn}}}(a_t \mid h_t) - \log \pi_{\theta_n}^{\mathsf{K}_{\text{roll}}}(a_t \mid h_t)}_{\Delta_{t,n}^{\text{kernel}}} + \underbrace{\log \pi_{\theta_n}^{\mathsf{K}_{\text{roll}}}(a_t \mid h_t) - \log \pi_{\theta_v}^{\mathsf{K}_{\text{roll}}}(a_t \mid h_t)}_{\Delta_{t,n,v}^{\text{stale}}}. \tag{2.1}$$

When $n = v$, parameter staleness vanishes, but the kernel term generally does not. A fully synchronous system can therefore still be off-policy.

What would make softmax training literally on-policy? One sufficient route is to replay the policy-visible sequence through the same prompt-state construction, recurrent KV-cache update, kernels, precision, and sampling semantics used during rollout, while propagating gradients through the recurrently constructed cache. Reusing a detached actor cache omits part of the score derivative. A second route is a parallel implementation whose output distribution has been verified to match the rollout path under the declared execution configuration. Without one of these contracts, "same checkpoint" is not an on-policy guarantee.

3 Linear Attention and State-Space Models

For linear attention and state-space models, the distinction is structural. Inference is naturally expressed as a recurrence

$$S_t = F_\theta(S_{t-1}, x_t), \qquad o_t = G_\theta(S_t, x_t), \tag{3.1}$$

while efficient training often uses a parallel scan, a convolutional dual, or a chunkwise factorization.

For ordinary linear attention, the chunk size C interpolates between execution schedules:

$$C = 1 \iff \text{fully recurrent schedule}, \qquad C = L \iff \text{fully parallel schedule}. \tag{3.2}$$

Intermediate chunk sizes trade sequential state propagation for matrix-multiplication throughput (Yang et al., 2024a,b). The recurrent, scan, and chunkwise formulas can be algebraically equivalent, yet still implement different finite-precision operators. Changing the chunk size, scan tree, state materialization points, or accumulation precision changes the parenthesization of the recurrence. A small difference in one intermediate state can alter every later state and every later token probability.

This is why learner-side parallel scoring is not automatically the probability distribution that generated a recurrent rollout. A recurrent actor paired with a chunkwise learner must be treated as two effective policies unless parity has been established for the exact chunk schedule, precision, parallel layout, and prompt-state construction used in the run.

same recurrence on paper; different parenthesization, rounding, and state materialization in finite precision

Figure 2 Chunking changes the execution schedule. Algebraic equivalence across the line does not imply equality of the induced token distributions.

4 Why DeltaNet Is Especially Sensitive

DeltaNet (Schlag et al., 2021) makes schedule-dependent state error particularly sharp because the state update is a data-dependent corrective write. In one common orientation,

$$S_t = S_{t-1}\left(I - \beta_t k_t k_t^\top\right) + \beta_t v_t k_t^\top . \tag{4.1}$$

Efficient DeltaNet training does not execute Eq. (4.1) token by token. It rewrites the recurrence as ordered products of generalized Householder factors and uses a chunkwise WY-style representation to parallelize over sequence length (Yang et al., 2024b). The factors generally do not commute, so temporal order must be preserved algebraically. Even when the algebra preserves that order, chunkwise factorization changes floating-point parenthesization and the locations at which values are rounded.

The discrepancy then feeds back into the next update. An error in S_{t-1} changes the read $S_{t-1}k_t$. That changes the value erased by the delta rule, which changes S_t, which changes the next read and corrective write. The state error is therefore not merely observed downstream; it participates in the future update dynamics. Recurrent DeltaNet rollout and learner scoring with the parallelized delta-rule kernel must be regarded as different effective policies unless finite-precision parity is explicitly demonstrated. The same warning applies to gated delta-rule variants and other recurrent architectures whose training kernel changes the state-update schedule.

5 The Reinforcement-Learning Consequence

The safe default is to define the policy by both parameters and execution operator:

$$\text{effective policy} = \text{parameters} + \text{execution path}. \tag{5.1}$$

This leads to three implementation rules.

1. **Capture behavior probabilities from the actor path.** Store the log-probability emitted by the actual sampling engine at the point where each token is drawn. A learner-side replay is an acceptable substitute only after distributional parity with the actor path has been established.

2. **Do not fold kernel mismatch into parameter staleness.** Log the rollout-to-learner probability gap at fixed weights separately from version lag. Otherwise actor refresh may appear to solve a mismatch that actually comes from execution kernels.

3. **Reserve "on-policy" for a measured execution contract.** Literal on-policy learning requires either the same recurrent path for rollout and differentiable learner replay, or a parallel path verified to induce the same token distribution under the declared batch shape, precision, cache construction, chunk schedule, and parallel layout.

When those conditions are not met, off-policy is the correct systems abstraction even if rollout and learner use the same nominal checkpoint. In particular, replacing the actor probability in the denominator with a learner-engine recomputation discards the actual behavior distribution and destroys the intended change-of-measure interpretation. A minimal fixed-weight parity audit is given in Sec. A.

6 Solution I: Treat Kernel Mismatch as Off-Policy Data

The most general remedy is not to pretend that two executable paths are the same policy. Instead, define the behavior distribution by the path that actually sampled each token, preserve that probability in the rollout record, and perform an explicit change of measure at the learner.

6.1 A time-dependent behavior operator

A rollout need not use one operator for the whole response. For an ordinary softmax-attention actor, a useful abstraction is

$$\mathsf{K}_{\mathrm{beh},t} = \begin{cases} \mathsf{K}_{\mathrm{prefill}}, & t = 1, \\ \mathsf{K}_{\mathrm{decode}}, & t \geq 2, \end{cases} \tag{6.1}$$

where $t = 1$ denotes the first response token after prompt processing. For a linear-attention or state-space actor, $\mathsf{K}_{\mathrm{beh},t}$ may instead identify a recurrent state update, a fixed chunk boundary, or a particular scan schedule. Let

$$\mu_{v,t}(\cdot \mid h_t) := \pi_{\theta_v}^{\mathsf{K}_{\mathrm{beh},t}}(\cdot \mid h_t) \tag{6.2}$$

be the behavior distribution of a record generated at actor version v. The learner at version n differentiates

$$p_{n,t}(\cdot \mid h_t) := \pi_{\theta_n}^{\mathsf{K}_{\mathrm{learn}}}(\cdot \mid h_t). \tag{6.3}$$

The recordwise importance ratio is therefore

$$\rho_{t,n,v} = \frac{p_{n,t}(a_t \mid h_t)}{\mu_{v,t}(a_t \mid h_t)} = \exp\!\left(\ell^{p}_{t,n} - \ell^{\mu}_{t,v}\right), \tag{6.4}$$

where $\ell^{\mu}_{t,v}$ must be captured from the actor path at sampling time. Recomputing it with $\mathsf{K}_{\mathrm{learn}}$ makes the denominator a different policy and erases the correction one intended to perform.

6.2 Prefill–decode and chunkwise–recurrent correction are the same change of measure

At fixed learner parameters, the kernel component can be factored through any sequence of executable bridge operators $\mathsf{K}_0, \ldots, \mathsf{K}_M$, with $\mathsf{K}_0 = \mathsf{K}_{\mathrm{beh},t}$ and $\mathsf{K}_M = \mathsf{K}_{\mathrm{learn}}$:

$$\frac{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{learn}}}(a_t \mid h_t)}{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)} = \prod_{j=1}^{M} \frac{\pi_{\theta_n}^{\mathsf{K}_j}(a_t \mid h_t)}{\pi_{\theta_n}^{\mathsf{K}_{j-1}}(a_t \mid h_t)}. \tag{6.5}$$

For softmax attention, one bridge factor may compare teacher-forced parallel scoring with recurrent KV-cached decode. For a state-space layer, another may compare a chunkwise-parallel kernel with the tokenwise recurrent update. These factors are diagnostics and may be useful for attribution, but they must telescope to the single learner-to-actor ratio in Eq. (6.4); multiplying independently defined "corrections" that do not share the same intermediate distributions double-counts the mismatch.

Separating kernel and version effects gives

$$\rho_{t,n,v} = \underbrace{\frac{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{learn}}}(a_t \mid h_t)}{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)}}_{\rho_{t,n}^{\mathrm{kernel}}} \underbrace{\frac{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)}{\pi_{\theta_v}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)}}_{\rho_{t,n,v}^{\mathrm{stale}}}. \tag{6.6}$$

The first term remains at zero parameter lag; the second remains even when actor and learner use the same kernel but different model versions.

6.3 What token importance weighting corrects

Consider a finite-horizon trajectory $\tau = (h_1, a_1, r_1, \ldots, h_H, a_H, r_H)$ and define

$$G_t(\tau) = \sum_{u=t}^{H} r_u, \qquad J_{\mathsf{K}_{\mathrm{learn}}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta^{\mathsf{K}_{\mathrm{learn}}}} \left[\sum_{t=1}^{H} r_t \right]. \tag{6.7}$$

Let d_μ^t be the history distribution at step t under the behavior process and let

$$Q_t^\mu(h, a) = \mathbb{E}_{\tau \sim \mu}[G_t \mid h_t = h, a_t = a]. \tag{6.8}$$

Holding the behavior occupancy and continuation fixed, define the local one-step-deviation surrogate

$$\mathcal{S}_\mu(\theta) = \sum_{t=1}^{H} \mathbb{E}_{h \sim d_\mu^t} \mathbb{E}_{a \sim \pi_\theta^{\mathsf{K}_{\mathrm{learn}}}(\cdot \mid h)}[Q_t^\mu(h, a)]. \tag{6.9}$$

Under the support condition

$$\pi_\theta^{\mathsf{K}_{\mathrm{learn}}}(\cdot \mid h) \ll \mu_t(\cdot \mid h) \tag{6.10}$$

for every admitted behavior history, its gradient is

$$\nabla_\theta \mathcal{S}_\mu(\theta) = \mathbb{E}_{\tau \sim \mu} \left[\sum_{t=1}^{H} \rho_t G_t \nabla_\theta \log \pi_\theta^{\mathsf{K}_{\mathrm{learn}}}(a_t \mid h_t) \right]. \tag{6.11}$$

Thus a token ratio exactly corrects the conditional action law at histories sampled by the actor. It does *not* replace d_μ^t by the target-policy history distribution.

A second distinction is equally important: importance weighting changes the sampling measure, not the score Jacobian. Equation (6.11) is exact for the declared target $\pi_\theta^{\mathsf{K}_{\text{learn}}}$ because its numerator and score both use $\mathsf{K}_{\text{learn}}$. If the intended scientific target is instead the recurrent deployment policy $\pi_\theta^{\mathsf{K}_{\text{recur}}}$, then the correct coefficient and score are

$$\rho_t^{\text{recur}} = \frac{\pi_\theta^{\mathsf{K}_{\text{recur}}}(a_t \mid h_t)}{\mu_t(a_t \mid h_t)}, \qquad s_t^{\text{recur}} = \nabla_\theta \log \pi_\theta^{\mathsf{K}_{\text{recur}}}(a_t \mid h_t). \tag{6.12}$$

A recurrent probability ratio multiplied by a parallel-kernel score is not generally the gradient of either policy. Importance sampling therefore solves actor-to-target distribution mismatch only when the learner can differentiate the declared target operator. Otherwise the update is a cross-operator surrogate, and literal optimization of the recurrent deployment policy requires differentiable recurrent replay or a forward-and-backward equivalent kernel.

If exact prefix occupancy is required, define

$$W_t = \prod_{u=1}^{t} \rho_u; \tag{6.13}$$

if exact full-trajectory change of measure is required, use W_H. For a terminal reward,

$$\nabla J_{\mathsf{K}_{\text{learn}}}(\theta) = \mathbb{E}_{\tau\sim\mu}\left[W_H R(\tau) \sum_{t=1}^{H} \nabla_\theta \log \pi_\theta^{\mathsf{K}_{\text{learn}}}(a_t \mid h_t)\right], \tag{6.14}$$

but the variance of W_t or W_H can be prohibitive. Token-local correction is therefore a deliberate local surrogate, not a claim of full on-policy trajectory correction.

A practical score loss is

$$\mathcal{L}_{\text{IS}}(\theta) = -\frac{1}{\sum_i \omega_i} \sum_i \omega_i \sum_t m_{i,t}\, \text{stopgrad}(\rho_{i,t} A_{i,t}) \log \pi_\theta^{\mathsf{K}_{\text{learn}}}(a_{i,t} \mid h_{i,t}), \tag{6.15}$$

where $m_{i,t}$ selects policy tokens and $A_{i,t}$ is a return or a valid action-independent-baseline advantage. The behavior log-probability in $\rho_{i,t}$ remains immutable across all learner steps.

6.4 Support, variance, and finite-step control

Hard top-k, top-p, min-p, or similar truncation can set $\mu(a \mid h) = 0$ while the learner assigns positive mass. Then Eq. (6.10) fails and no finite ratio recovers the missing actions. Exact mode can instead use a shared full-support transform

$$\mathcal{F}_{T,\varepsilon}(z)(a) = (1-\varepsilon)\frac{\exp(z_a/T)}{\sum_b \exp(z_b/T)} + \varepsilon u(a), \qquad u \in \Delta(\mathcal{A}), \quad u(a) > 0, \tag{6.16}$$

for actor sampling, stored probabilities, learner scoring, differentiation, and divergence measurement. A hard-truncated run must be labeled as a support-truncated approximation or redefine the learner target on the same stored support.

Importance ratios should not be clipped merely to disguise kernel mismatch: clipping biases Eq. (6.11). Finite update size should be controlled separately by a declared behavior-to-learner divergence and candidate-step acceptance, backtracking, or early stopping. Ratio variance still matters because

$$\mathbb{E}_{a\sim\mu}[\rho(a)^2] = 1 + \chi^2\left(\pi_\theta^{\mathsf{K}_{\text{learn}}} \middle\| \mu\right), \qquad \chi^2(p\|m) = \sum_a \frac{(p(a)-m(a))^2}{m(a)}. \tag{6.17}$$

A production implementation should report log-ratio quantiles, $\mathbb{E}[\rho^2]$, effective sample size, nonfinite ratios, and full or reduced KL/TV diagnostics. Off-policy correction is mathematically clean only when the behavior probability and support metadata are trustworthy.

7 Solution II: Make One Execution Rule Canonical

Importance weighting preserves throughput flexibility, but it accepts that rollout and learner are different policies. The alternative is to remove the operator gap by defining one executable recurrence or chunk rule as the policy itself.

7.1 Canonical chunkwise updates

Let the accepted token stream be divided into blocks $X_j = x_{jC+1:(j+1)C}$. A chunkwise state machine can be written as

$$O_j = G_\theta^{(C)}(S_j, X_j), \qquad S_{j+1} = F_\theta^{(C)}(S_j, X_j). \tag{7.1}$$

Literal alignment requires actor and learner to use the same chunk size C, boundary convention, within-chunk causal mask, state read-before-write convention, optimizer-like memory update, precision, and state-materialization points. TTT layers treat the hidden state as a model updated by a self-supervised learning step, and their systems formulation uses mini-batch or chunkwise updates to expose matrix multiplication. Titans similarly chunks sequences and updates a neural long-term memory between segments (Sun et al., 2024; Behrouz et al., 2025). These architectures show that a chunk is allowed to be part of the *model semantics*, rather than only an implementation optimization.

This route removes mismatch only when training and deployment share that same semantic rule. It is not enough that both implementations are called "TTT", "Titans", or "chunkwise". Changing C, carrying momentum across different boundaries, reading a pre-update state in one path and a post-update state in the other, or replacing a serial update by a different scan tree creates a new effective policy.

There is also an autoregressive limitation. A chunkwise memory update can consume a block of *already accepted* tokens at once, but the tokens inside a stochastic language-model block are not known before they are generated. Chunkwise state semantics do not by themselves permit exact parallel token sampling. Parallel proposals require a separate speculative mechanism, developed in Sec. 9.

7.2 Pure recurrent training and inference

The strongest equality contract is to use the same tokenwise recurrence everywhere:

$$S_t = F_\theta(S_{t-1}, x_t), \qquad z_t = G_\theta(S_t, x_t), \qquad a_t \sim \mathcal{T}(z_t). \tag{7.2}$$

Execution choice	Policy-equality condition	Main benefit	Main cost / caveat
Learner parallel, actor recurrent	None assumed; use recordwise IS	Highest learner throughput	Local off-policy objective; ratio variance
Canonical chunkwise rule	Same C, boundaries, update semantics, and precision	Chunk-level matmul efficiency	Does not parallelize unknown autoregressive tokens
Pure recurrent rule	Same tokenwise path with gradients through state	Strongest literal on-policy contract	Serial replay and activation memory
Verified parallel kernel	Measured equality to recurrent path	Parallelism without IS	Proof is configuration-specific and fragile

Table 1 Execution alignment trades throughput flexibility for a stronger policy-equality contract.

The rollout actor, differentiable learner replay, validation, and deployment all execute Eq. (7.2) in the same order and precision. Gradients must pass through the learner-constructed recurrent states; replaying a detached actor cache omits part of the score derivative.

RWKV-7 is a useful architectural example because it has constant-memory, constant-time recurrent inference per token. However, RWKV-7 also explicitly retains parallelizable training (Peng et al., 2025). Therefore "use RWKV-7" is not by itself a parity guarantee. To obtain the pure-RNN contract, the learner must intentionally replay the recurrent form, or the parallel training kernel must be verified to match that recurrent form under the declared numerical configuration.

Pure recurrent replay sacrifices sequence parallelism and can make long-response RL expensive. Truncated backpropagation, checkpointing, or state detachment may recover memory or throughput, but each changes either the objective or the gradient contract. The benefit is conceptual clarity: when the exact same recurrence is differentiated, there is no prefill–decode or chunkwise–recurrent importance factor at fixed parameters.

8 Solution III: Use Higher Precision for Recurrent State

Linear attention and state-space layers repeatedly feed a stored state back into the next update. This creates a different numerical risk from a feed-forward layer whose local rounding error is observed once. Let S_t be the reference recurrence and $\widehat{S}_t$ the implemented state. With $e_t = \widehat{S}_t - S_t$, a generic bound is

$$\|e_t\| \le L_t \|e_{t-1}\| + \delta_t^{\mathrm{round}} + \delta_t^{\mathrm{schedule}}, \tag{8.1}$$

where L_t is a local state-sensitivity factor, $\delta_t^{\mathrm{round}}$ is arithmetic error under a fixed schedule, and $\delta_t^{\mathrm{schedule}}$ is the difference introduced by changing parenthesization, scan tree, or chunk materialization. Unrolling gives

$$\|e_t\| \le \sum_{u=1}^{t} \left(\prod_{j=u+1}^{t} L_j \right) \left(\delta_u^{\mathrm{round}} + \delta_u^{\mathrm{schedule}} \right). \tag{8.2}$$

Higher precision reduces δ^{round} and can substantially delay state drift. It does not remove $\delta^{\mathrm{schedule}}$, and it does not prevent amplification when the product of sensitivities is large.

A practical precision ladder is:

1. **FP32 state by default.** Store recurrent SSM/linear-attention states, normalization statistics, decay products, and state-update accumulations in FP32 even when embeddings, MLPs, projections, and communication use BF16 or FP16.
2. **FP32 sensitive parameters and activations when needed.** Keep recurrence-defining parameters, gates, time constants, and the input/output operands of state updates in FP32. The official Mamba implementation notes that higher precision for main parameters may be necessary because SSMs are sensitive to recurrent dynamics (Gu and Dao, 2023; State Spaces Team, 2026).
3. **FP64 as a reference path.** Use FP64 recurrent states and updates for small-model audits, boundary-state canonicalization, or sampled production checks. Full-model FP64 training is usually too expensive to be the default.

For DeltaNet-like updates, at minimum compute the read $S_{t-1}k_t$, the erase residual $v_t - S_{t-1}k_t$, and the rank-one write accumulation in FP32. If the state remains unstable, promote the state and recurrence-defining weights to FP64 in the audit path. In a correctness-first experiment, all linear-attention/SSM-layer inputs, gates, weights, intermediate activations, and cache/state can be evaluated in FP64 while unrelated embedding and MLP blocks remain in lower precision. Casting an already corrupted BF16 state to FP32 does not recover lost information; precision must be applied before the recurrent accumulation.

Higher precision is a mitigation, not an on-policy certificate. Two FP64 programs with different reduction trees can still disagree, and small logit differences can matter under low temperature or near sampling boundaries. Every precision change must therefore be paired with the fixed-weight parity audit in Sec. A.

9 Solution IV: Prefill or Parallel Proposal with Recurrent Verification

A fourth route separates *speed* from *policy semantics*. Let the recurrent implementation define the canonical target distribution

$$p_i(\cdot) = \pi_\theta^{\mathsf{K}_{\mathrm{recur}}}(\cdot \mid h_i), \tag{9.1}$$

and let a faster prefill, chunkwise, approximate, or auxiliary path define a proposal

$$q_i(\cdot) = \pi_\theta^{\mathsf{K}_{\mathrm{prop}}}(\cdot \mid h_i). \tag{9.2}$$

The proposal drafts a block $y_{1:\gamma}$. The recurrent path verifies that block and uses modified rejection sampling. For each proposed token,

$$\alpha_i(y_i) = \min\left(1, \frac{p_i(y_i)}{q_i(y_i)}\right). \tag{9.3}$$

If a proposal is rejected, sample the replacement token from

$$r_i(a) = \frac{[p_i(a) - q_i(a)]_+}{\sum_b [p_i(b) - q_i(b)]_+}. \tag{9.4}$$

This is the exact speculative-sampling correction used to preserve the target distribution (Leviathan et al., 2023; Chen et al., 2023). An argmax agreement test, a logit-tolerance test, or ordinary resampling from p_i after arbitrary acceptance does not in general preserve stochastic target sampling.

Algorithm 1 Parallel/prefill proposal with recurrent target verification.

Require: Committed recurrent state S; proposal operator K_{prop}; recurrent target $\mathsf{K}_{\text{recur}}$; draft length γ.

The proposal engine produces candidate tokens $y_{1:\gamma}$ and categorical distributions $q_1, \dots, q_\gamma$.
Starting from a temporary copy of S, run the recurrent target along the candidate prefix to obtain $p_1, \dots, p_\gamma$ and temporary states.
for $i = 1, \dots, \gamma$ **do**
    Draw $u_i \sim \text{Uniform}(0, 1)$.
    if $u_i \leq \min(1, p_i(y_i)/q_i(y_i))$ **then**
        Commit y_i and the corresponding recurrent state transition.
    else
        Draw $z \sim \text{Normalize}([p_i - q_i]_+)$; commit z through the recurrent target; discard later speculative states; **stop**.
    end if
end for
If every draft token is accepted, optionally draw one additional token from the next recurrent target distribution.

The phrase "use the prefill kernel as speculative decoding" needs one qualification. A teacher-forced prefill kernel can score a supplied candidate block in parallel, but by itself it cannot invent later autoregressive tokens: the logits for position i require candidate tokens before i. A valid proposal source must therefore exist, for example, a smaller draft model, a same-model approximate recurrence, a Jacobi or masked parallel proposal, a candidate tree, or a previous speculative iterate. The prefill kernel can then evaluate or refine that proposal. Exactness additionally requires access to the proposal probabilities needed by Eqs. (9.3) and (9.4).

This design reverses the usual performance emphasis. Standard speculative decoding often uses a cheap autoregressive drafter and a parallel target verifier. Here the parallel or prefill path is the drafter, while the recurrent code is the correctness authority. It can still be useful when recurrent state transitions are cheap, the draft has a high acceptance rate, and several candidate paths can be batched. It may provide little or no speedup when recurrent verification remains the dominant serial cost.

The speculative acceptance ratio is not the RL importance ratio. Once Alg. 1 is exact, accepted tokens are distributed according to p_i, so the rollout record stores $\log p_i(a_i)$ as its behavior probability. The draft probability q_i is proposal metadata, and proposal-only cache or memory state is never committed. If the learner later differentiates a parallel or chunkwise policy different from p_i, the RL update still needs Eq. (6.4). If the recurrent policy itself is the target, the learner also needs the recurrent score in Eq. (6.12); an importance coefficient alone does not replace recurrent differentiation.

10 Recommended Hybrid Contract

The four solutions are complementary rather than mutually exclusive. A robust production design is:

1. Declare a canonical recurrent actor/deployment policy and capture its per-token log-probability at sampling time.
2. Allow the learner to use a fast parallel or chunkwise operator only after declaring whether it is the target policy or an approximation to the recurrent target. For a learner-defined target,

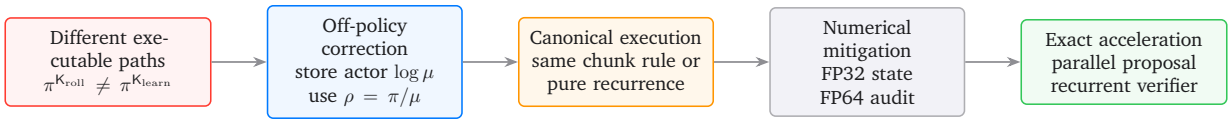

Figure 3 A solution stack. Importance correction provides the general fallback; execution alignment removes the gap when affordable; higher precision reduces state drift; speculative verification can recover speed without changing the recurrent target distribution.

apply the unclipped recordwise ratio under an explicit support contract; for a recurrent target, differentiate recurrent replay or a verified forward-and-backward equivalent kernel.

3. Keep recurrent states and state-update accumulations in FP32; maintain an FP64 reference path for parity audits and selected boundary-state checks.
4. Optionally accelerate rollout with a parallel proposal, but let exact recurrent speculative verification determine accepted tokens and behavior probabilities.
5. Control finite learner movement with a separate behavior-anchored divergence statistic; do not ask ratio clipping to serve simultaneously as change of measure and trust region.

Remedy	**Equality status**	**What it solves**	**Residual limitation**
Recordwise importance correction	Off-policy by design	Prefill/decode, chunk/recurrent, and version mismatch	Corrects actions at behavior histories unless prefix/sequence ratios are used
Canonical chunkwise rule	On-policy if execution contract matches	Makes chunk boundaries and state updates part of model semantics	Unknown autoregressive tokens remain sequential
Pure recurrent replay	On-policy at fixed weights	Removes parallel/recurrent operator gap	Lower training throughput and higher activation cost
FP32/FP64 state	No equality guarantee	Reduces recurrent numerical drift	Cannot remove schedule error or unstable dynamics
Parallel draft + recurrent verification	Exact recurrent sampling if modified rejection is exact	Uses fast proposal without changing actor distribution	Needs valid proposal probabilities and sufficient acceptance rate

Table 2 No single remedy dominates every regime. The correct choice depends on whether the objective is exact policy equality, scalable local off-policy learning, numerical stability, or rollout latency.

11 Conclusion

The prefill–decode mismatch is not resolved by synchronizing model weights. Standard rollout and learner pipelines evaluate response-token probabilities with different executable operators: recurrent decode on the actor and parallel teacher forcing on the learner. Linear attention and state-space models add a second axis, because recurrent, scan, and chunkwise schedules can induce different finite-precision state trajectories. Delta-rule updates can amplify those differences by feeding state error back into later corrective writes.

There are four principled responses. The most general is recordwise off-policy learning with the actual actor probability in the denominator. A stronger but more expensive response is to make

one chunkwise or recurrent rule canonical across rollout and differentiable learner replay. Higher-precision recurrent state reduces drift but does not prove equality. A parallel or prefill path can also be demoted to a speculative proposal, with exact recurrent rejection sampling defining the accepted policy. In every case, the executable distribution—not the checkpoint name—is the mathematical policy.

References

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2025.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, Nathan Wilce, Johan S. Wind, Tianyi Wu, Daniel Wuttke, and Christian Zhou-Zheng. RWKV-7 "Goose" with expressive dynamic state evolution. arXiv preprint arXiv:2503.14456, 2025.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, pages 9355–9366, 2021.

State Spaces Team. Mamba reference implementation: precision notes. https://github.com/state-spaces/mamba, accessed August 7, 2026.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. arXiv preprint arXiv:2407.04620, 2024.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems 37*, 2024.

Ziyang Zhang, Xinheng Ding, Jiayi Yuan, Rixin Liu, Huizi Mao, Jiarong Xing, and Zirui Liu. Deterministic inference across tensor parallel sizes that eliminates training–inference mismatch. arXiv preprint arXiv:2511.17826, 2025.

Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, and Xiao Yu. Diagnosing training inference mismatch in LLM reinforcement learning. arXiv preprint arXiv:2605.14220, 2026.

Appendix

A Minimal Fixed-Weight Parity Audit 16

B Record and Logging Checklist 16

C Minimal Off-Policy Record Contract 16

D Reference Pseudocode for Kernel-Corrected Learning 17

E Decision Checklist 18

A Minimal Fixed-Weight Parity Audit

A kernel-parity audit should be run independently of training. Freeze the model parameters, feed identical prefixes to the actual rollout and learner paths, and compare the token distributions before introducing parameter staleness. This separates the execution-operator gap from the ordinary actor–learner version gap.

Architecture	Executable comparisons
Softmax attention	Prompt prefill, recurrent KV-cached decode, differentiable recurrent replay, and full teacher-forced causal scoring.
Linear attention / SSM	Recurrent, scan, and chunkwise forms across chunk sizes, boundary conventions, and state-materialization schedules.
DeltaNet	Tokenwise recurrent delta-rule updates versus the parallelized chunkwise delta-rule kernel, including FP32 and FP64 reference states.
TTT / Titans-style memory	Online token updates versus the exact production mini-batch or segment update, with identical read-before-write and momentum semantics.
Speculative rollout	Proposal probabilities, recurrent verifier probabilities, acceptance decisions, residual samples, and final actor log-probabilities.

Table 3 The audit compares executable paths, not checkpoint names.

Vary precision, quantization, batch shape, tensor parallelism, and sequence parallelism one factor at a time. When full distributions are available, report KL divergence or total variation. Also report maximum and mean absolute log-probability error and sampled probability-ratio quantiles. The goal need not be bitwise equality; the goal is to determine whether the equality required by the claimed algorithm holds under the production execution contract.

B Record and Logging Checklist

For every sampled policy token, preserve the actor-path log-probability together with enough metadata to identify the effective behavior operator. At minimum, this includes the behavior checkpoint version, rollout engine, attention or recurrence form, prefill/decode mode, chunk size or scan schedule, cache construction, precision, quantization, sampling transform, tensor-parallel layout, and sequence-parallel layout.

The learner should log two distinct quantities:

$$\Delta^{\mathrm{kernel}}_{t,n} = \log \pi^{\mathsf{K}_{\mathrm{learn}}}_{\theta_n}(a_t \mid h_t) - \log \pi^{\mathsf{K}_{\mathrm{roll}}}_{\theta_n}(a_t \mid h_t), \tag{B.1}$$

$$\Delta^{\mathrm{stale}}_{t,n,v} = \log \pi^{\mathsf{K}_{\mathrm{roll}}}_{\theta_n}(a_t \mid h_t) - \log \pi^{\mathsf{K}_{\mathrm{roll}}}_{\theta_v}(a_t \mid h_t). \tag{B.2}$$

The first is an execution mismatch at fixed weights; the second is parameter staleness under a fixed rollout operator. Logging only their sum hides which systems intervention is needed.

C Minimal Off-Policy Record Contract

A rollout record is valid only if the learner can identify the distribution that produced every policy token. At minimum it contains:

1. prompt, policy-visible sequence, sampled token IDs, and policy-token mask;
2. actor-emitted ℓ_t^μ for every policy token and the behavior version v;
3. a per-token or segment-resolved execution signature: prefill/decode mode, recurrent or chunk-wise form, chunk size and boundary convention, state precision, parameter precision, quantization, cache construction, tensor/sequence parallel layout, and reduction mode;
4. sampling-transform metadata, including temperature, support mode, probability floor, grammar/safety mask identity, and any hard truncation;
5. for speculative rollout, proposal operator/version, q_i, recurrent verifier version, acceptance/rejection outcome, and the final recurrent behavior probability $p_i(a_i)$;
6. immutable reward, environment, verifier, termination, and ticket-accounting metadata.

The learner may append diagnostics but must never overwrite ℓ_t^μ with a learner-side recomputation.

D Reference Pseudocode for Kernel-Corrected Learning

Algorithm 2 One kernel-corrected learner step.

Require: Immutable records with actor log-probabilities ℓ^μ and support/execution metadata; current learner θ_n; declared divergence threshold.

Reject records whose support or execution metadata violate the run contract.
Evaluate current learner log-probabilities ℓ^p with $\mathsf{K}_{\mathrm{learn}}$.
On valid policy tokens, compute $\log \rho \leftarrow \ell^p - \ell^\mu$ and $\rho \leftarrow \exp(\log \rho)$ in FP64; reject nonfinite values rather than clipping them.
Compute the return or valid baseline advantage A and the score loss in Eq. (6.15).
Save model, optimizer, and gradient-scaler state; take a tentative optimizer step.
Recompute the declared behavior-to-learner divergence on the candidate parameters.
if the candidate violates the threshold **then**
    Restore all saved state; backtrack or reject the batch.
else
    Commit the step and log ratio moments, effective sample size, divergence, kernel gap, and version gap separately.
end if

E Decision Checklist

Question	Required action
Can rollout and learner execute the same recurrent rule with acceptable throughput?	Use recurrent differentiable replay and validate parity; no fixed-weight kernel ratio is then needed.
Is one chunk rule part of the intended model semantics?	Freeze chunk size, boundaries, read/write timing, momentum, precision, and masks across training and inference.
Must learner retain a faster parallel/chunkwise kernel?	Treat data as off-policy, store actor log-probabilities, enforce common support, and use recordwise importance correction.
Are recurrent states drifting?	Promote state/update arithmetic to FP32, test sensitive weights/activations in FP32, and compare against an FP64 audit path.
Can a fast path propose accurate blocks?	Use exact speculative acceptance and residual sampling with the recurrent path as target; store recurrent target probabilities for RL.
Does the proposal path only score supplied tokens?	Add a genuine candidate generator; a teacher-forced prefill evaluator alone is not a drafter.
Are ratios or divergences large?	Tighten actor lag, improve kernel parity, reduce learner step size, or reject the batch; do not silently replace or clip the denominator.

Table 4 A compact systems decision tree for choosing among the four remedies.